CS6600 - Computer Architecture

Assignment 3

Computer Science and Engineering Department
IIT Madras, Chennai, Tamil Nadu
Due Date: Sunday, September 27, 2026

Instructions: Extend the processor from Assignment 2 with a TAGE-SC-L branch predictor and the pipeline control required to execute along a predicted path. Your processor must recover from incorrect predictions without changing the program's architectural result. Implement the predictor, speculation and recovery logic, and the statistics needed to evaluate them.

Migrate your solution from Assignment 2 into the starter supplied for Assignment 3. Preserve its new interfaces and instruction metadata. Retain the instruction set, execution-unit timing, cache configuration, data-hazard rules, and in-order retirement from PA2.

1 Branch Predictor

Implement one integrated TAGE-SC-L predictor in `PredictorOne`, which occupies slot 0. Slots 1 and 2 are fixed shadow predictors supplied with the starter; leave them unchanged. The statistical corrector and loop predictor belong inside `PredictorOne`.

Read the original TAGE paper [1] and the TAGE-SC-L descriptions [2, 3]. State which TAGE-SC-L variant you implement. You may choose table sizes and other design parameters, but all three components must function and participate in prediction.

Every table in your design (TAGE's tagged components, the statistical corrector's tables, the loop predictor's table) must be direct-indexed: a hash of the lookup context selects one address, and lookup reads that one entry and compares its stored tag. Aliasing between unrelated contexts that hash to the same address is expected and is what the allocation and replacement policy exists to manage. A table implemented as a searched or fully associative collection over more than a small, fixed number of ways is not an acceptable substitute, regardless of the storage total it reports: it does not correspond to real hardware at that storage budget, and it eliminates aliasing behavior that is intrinsic to the design you are implementing.

1. **TAGE:** Implement the base predictor and tagged tables with geometric history lengths. Include provider and alternate selection, prediction counters, allocation, replacement, usefulness management, and history maintenance.
2. **Statistical corrector:** Implement the signed-counter tables, combination of their outputs, training conditions, and the decision to retain or override TAGE's prediction, following the selected paper.
3. **Loop predictor:** Implement loop identification, iteration and trip-count tracking, confidence, replacement, and selection of the loop prediction.

Provide a finite branch target buffer (BTB), or an equivalent structure, to identify branch PCs and retain targets. BEQ and BNE require direction prediction; J is unconditional. HALT is not a branch. Indirect branches, calls, and returns are outside the assignment's instruction set.

Use the predictor interface supplied in the starter. What each method is actually for:

Method	What it does, and when
`Predict`	This is where the real work happens. It is called during fetch, before the instruction has even been decoded, so all the predictor has to go on is the address. Using that address and whatever state it has built up from past branches, it decides whether this looks like a taken branch and where it would go. Whatever it returns has to be held onto and carried alongside the instruction, since two more checks happen against it later.
`ObserveInstruction`	Called once decode has actually looked at the instruction and knows what it really is. Its only job is a sanity check: if `Predict` guessed this address was a branch, but decode reveals it is an ordinary instruction, this is where that gets caught. It does not reconsider or improve the prediction, it only reports when the earlier "is this even a branch" guess turns out to have been wrong.
`Update`	Called once the branch has actually executed and its real outcome is known. This is where the predictor learns: it is handed the real outcome alongside the original guess from `Predict`, compares the two, and adjusts its internal state so it does better the next time it sees this branch. The counters used for grading, correct predictions, direction errors, target errors, are incremented here.
`Reset`	Wipes the predictor back to a blank slate: every internal table and every counter returns to its starting value.
`StorageBits`	Reports how much memory the design is spending, in bits, across every table and counter it keeps. This is what keeps the assignment honest about hardware cost, not just prediction accuracy.
`Metrics`	Hands back whatever counters `Predict`, `ObserveInstruction`, and `Update` have been keeping, so they can be reported and checked.

Retain the original prediction and its metadata until resolution or squash. Use this saved prediction to assess correctness. Document your history update and recovery policy.

2 Pipeline Integration

The starter provides prediction records, fetch and execution metadata, checkpoint interfaces, squash helpers, and statistics structures. Complete the speculation controller and connect these mechanisms to your pipeline from PA2.

Fetch and tracking. Use slot 0 to select the next fetch address. A predicted taken branch selects its target; otherwise fetch proceeds to $PC + 4$. Carry the original prediction with the instruction through the pipeline. Use dynamic sequence numbers to distinguish instruction instances, including repeated executions of the same PC. Maintain branch checkpoints and stall when their capacity is exhausted. Stalls must not create repeated checkpoints or repeat an instruction's observation callback.

Speculative execution. Younger arithmetic instructions may dispatch and execute before an older branch resolves, subject to hazard rules from PA2 and resource availability. Younger loads, stores, branches, jumps, and HALT must wait until the older branch resolves. Dispatch remains in program order; a blocked instruction cannot be bypassed.

Resolution. The CONTROL execution pipeline determines the actual branch direction and target. Two separate calls happen here, and they are not interchangeable: `BranchPrediction System::Resolve` compares the selected predictor's saved guess against the real outcome, drives predictor learning, and updates the predictor and branch metrics; `SpeculationController::Resolve` is the call that actually decides whether a redirect is needed, using its own stored checkpoint rather than the caller's copy of the prediction. Treat the second call's `Recovery Decision` as the authoritative signal for whether to squash, not the first call's return value, which exists for metrics only. Resolving a branch through `SpeculationController::Resolve` also releases that branch's own checkpoint as part of the call; do not additionally try to remove it through `RemoveYoungerCheckpoints`, which only discards checkpoints strictly younger than the sequence it is given and is meant for squash-time cleanup of wrong-path checkpoints, not for releasing the checkpoint of the branch that just resolved. An incorrect prediction requires a redirect to the actual next PC. Handle false branch identification as well as direction and target errors.

Squash and recovery. Preserve the resolving branch and all older instructions. Discard younger entries from fetch, IF/ID, execution pipelines, completion storage, reservations, and checkpoints. Invalidate outstanding wrong-path fetches so that their eventual responses cannot re-enter the pipeline. Release predictor metadata owned by discarded instructions. Restore speculative predictor history if your design updates it before resolution.

Completion and retirement. An instruction may finish execution while still speculative. Keep its result in completion storage until it can retire in order. A squashed instruction must never write an architectural register or memory, or terminate the program. A completed younger instruction is removed during recovery just like an unfinished one; architectural registers do not need to be rolled back. The restriction on speculative memory dispatch prevents an incorrect-path store from becoming visible before recovery.

Preserve the simulator's WB, MEM, EX, ID, IF ordering and its current/next-state discipline. A squash must not be undone by a later stage operation copying or advancing stale state. Test recovery both while younger operations are executing and after they have completed.

Timing. The supplied predictor interface is synchronous: prediction is performed during fetch, without a separately modeled predictor pipeline. The branch's CONTROL-stage latency is its resolution latency, not the predictor's latency. Measure performance using the simulator's cycle count, including stalls and recovery. Neither C++ execution time nor a student-declared predictor latency is added to this count as predictor cycles.

3 Statistics and Evaluation

Maintain the existing processor, predictor, and speculation metrics, and expose them through the statistics interface. Count events where they occur. A stalled operation must not be counted repeatedly as an accepted fetch, dispatch, update, or squash. Reset must clear the measurements.

The measurements must distinguish:

- Execution: total cycles, retired instructions, speculative fetches and dispatches, and recovery cycles.
- Prediction: lookups, branch-identification hits and misses (this is what "BTB hits and misses" refers to elsewhere in this document – the branch target buffer and the branch-found/not-found outcome are the same event, not two separate counters), correct predictions, direction errors, target errors, and false branch identifications.
- Recovery: resolved branches, redirects, maximum speculation depth, and discarded fetches, execution operations, and completions.

The counters above are backed by fields already declared in the supplied interface (`PredictorMetrics`, `SpeculationMetrics`); populate them correctly rather than inventing replacements. Beyond these, your report must also characterize component-level behavior that the supplied interface has no dedicated field for: how often each part of TAGE actually provides the prediction, how often the statistical corrector overrides TAGE and whether that override was correct, and how often the loop predictor is selected and whether it was correct. Track these with your own instrumentation inside your predictor; there is no requirement to route them through `PrintStatistics`, and no fixed format for reporting them, but your report must present actual measured numbers, not a qualitative description.

Preserve existing counter names. Document each counter's event definition, including whether it counts all lookups or only predictions that are later resolved. Explain how recovery intervals are measured. Report IPC and branch mispredictions per thousand retired instructions:

$$\mathrm{IPC} = \frac{I_{\mathrm{retired}}}{C_{\mathrm{cycles}}}, \qquad \mathrm{MPKI} = 1000\frac{M_{\mathrm{direction}} + M_{\mathrm{target}}}{I_{\mathrm{retired}}}.$$

Count target errors separately from direction errors so that one branch is not counted twice. Report false branch identifications separately. Check that resolved-branch totals agree with their outcome categories and that redirects agree with the corresponding recovery events.

Lightweight implementation. Marks will be awarded for implementation cost as well as performance. Report the provisioned logical storage of every predictor structure, including tags, counters, targets, histories, confidence and replacement state. Give the number of entries, bits per field, and total bits. Include shared state once, and report retained prediction metadata and checkpoint storage separately. Declared field widths must match the ranges enforced by the implementation.

Report table reads and writes for each predictor component to show the work performed per prediction and update. Keep any host runtime and memory measurements separate from simulated cycles and logical predictor storage. These measurements do not constitute physical area, power, or timing estimates.

Correctness. Compare final registers, architectural memory, and termination against a trusted execution of the same program. The supplied `Machine::PrintState` already accounts for dirty cache contents when reporting memory – it overlays pending writebacks and cache-resident lines onto the backing image before printing, so use it, or match its behavior exactly, rather than reading backing memory directly. Test direction and target errors, changing loop behavior, wrong-path completion, wrong-path HALT, and fetch responses that arrive after recovery. Use

the counters to check event accounting and diagnose failures; consistent counters alone do not establish correctness.

Evaluation will consider correctness, completeness of the predictor and pipeline integration, simulated execution performance, implementation cost, and the experimental report. Performance comparisons will use common programs and machine configurations. Reducing cycle counts by dropping work, omitting components, or changing timing from PA2 does not satisfy the assignment.

4 Submission

Submit the complete source tree, test programs with expected results, commands needed to reproduce your experiments, and a PDF report. The report must contain:

1. Team members and their contributions.
2. The selected paper variant, predictor organization, parameters, and pipeline integration.
3. **All assumptions and departures from the papers or starter specification**, with their justification and effect on the design.
4. Correctness tests, measured cycle counts and prediction statistics, and an explanation of the results.
5. Storage calculations, component access counts, and the trade-offs made to keep the implementation lightweight.
6. References for the papers and any other material used.

A possible division among three students is TAGE and history management, the statistical corrector, and the loop predictor. Speculation, recovery, testing, and final integration remain the responsibility of the whole team.

References

[1] André Seznec and Pierre Michaud. *A case for (partially) TAgged GEometric history length branch prediction.* Journal of Instruction-Level Parallelism, vol. 8, 2006. `https://jilp.org/vol8/v8paper1.pdf`.

[2] André Seznec. *TAGE-SC-L Branch Predictors.* 4th Championship Branch Prediction, 2014. `https://www.jilp.org/cbp2014/paper/AndreSeznec.pdf`.

[3] André Seznec. *TAGE-SC-L Branch Predictors Again.* 5th Championship Branch Prediction, 2016. `https://www.jilp.org/cbp2016/paper/AndreSeznecLimited.pdf`.

5 Submission Instructions

Submit the complete Assignment 3 starter with your implementation inside your existing `pipesim` repository in a new top-level folder named `submission3` (all lowercase), alongside `assignment 1` and `asignment 2`. Use the folder name `submission3` exactly as shown. Do not modify `assignment 1` or `asignment 2`.

Place the complete source tree, tests with expected results, reproduction commands, and `report.pdf` inside `submission3`. Preserve the starter's directory structure within this folder. Commit and push your submission to the existing repository by Sunday, September 27, 2026.

Each contributor's changes must be identifiable in the repository's Git history. Use commits attributed to the contributor who made the changes; jointly authored commits must identify all contributors. Preserve this attribution in the submitted history. Contributions that cannot be attributed through the Git history will not be considered for individual credit. Listing a contribution in the report alone is insufficient.